Docker + Nginx Restaurant Website Deployment

Project Documentation

Overview

This project demonstrates deploying a static restaurant website using Docker and Nginx on an Ubuntu EC2 instance.

Prerequisites

- Ubuntu EC2 instance
- Docker installed
- SSH access
- Public IP
- Restaurant HTML page

Implementation Steps

Update System

```
sudo apt update
sudo apt install -y docker.io
```

Start Docker

```
sudo systemctl start docker
sudo systemctl enable docker
```

Verify Docker

```
docker --version
docker ps
```

Create Project

```
mkdir -p ~/restaurant-project
cd ~/restaurant-project
```

Pull Nginx

```
docker pull nginx
```

Create Volume

```
docker volume create restaurant-data
```

Run Container

```
docker run -d \
  --name restaurant-web-v2 \
  -p 8080:80 \
  -v restaurant-data:/usr/share/nginx/html \
  nginx
```

Create Website

```
vi index.html
```

Copy Website

```
docker cp index.html restaurant-web-v2:/usr/share/nginx/html/index.html
```

Create Images Folder

```
mkdir -p ~/restaurant-project/images
```

Upload Image (Mac)

```
scp -i docker-instance-1.pem restaurant.jpg
ubuntu@<EC2_PUBLIC_IP>:/home/ubuntu/restaurant-project/images/
```

Copy Image

```
docker exec restaurant-web-v2 mkdir -p /usr/share/nginx/html/images

docker cp ~/restaurant-project/images/restaurant.jpg restaurant-web-
v2:/usr/share/nginx/html/images/
```

Verify

```
docker exec restaurant-web-v2 ls /usr/share/nginx/html/images
```

Access

```
Website: http://<EC2_PUBLIC_IP>:8080
Image: http://<EC2_PUBLIC_IP>:8080/images/restaurant.jpg
```

Validation

- docker ps shows container running
- Website loads in browser
- Image is accessible via /images path

Best Practices

- Use bind mounts for development
- Store images in an images folder

- Use reverse proxy and HTTPS in production
- Keep Docker images updated

Cleanup

docker stop restaurant-web-v2
docker rm restaurant-web-v2
docker volume rm restaurant-data

Author: Akash Kolhe